

Genericidad

La generalización consiste en abstraer lo que es común a un conjunto de entidades para definir un concepto que las incluya a todas. La generalización es una estrategia fundamental en la resolución de problemas de diferentes disciplinas, en particular en Ciencias Exactas, Ciencias Naturales y en Ciencias de la Computación. El concepto de **genericidad**, muy relacionado con la estrategia de generalización, se refiere a todo aquello que es común entre los miembros de una especie y permite por lo tanto establecer un patrón común.

En Matemática, una fórmula genérica es aquella que comprende a un número grande de casos y puede usarse para obtener fórmulas particulares. Por ejemplo, una ecuación de segundo grado tiene cero, una o dos soluciones, que pueden calcularse a partir de la siguiente fórmula genérica:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

También se usa el término fórmula genérica para expresar una propiedad que se cumple siempre. Por ejemplo:

$$a^m \cdot a^n = a^{m+n}$$

En Farmacología un medicamento genérico es aquel que no se distribuye con un nombre comercial y posee la misma concentración y dosificación que otro de marca. Los medicamentos genéricos son bioequivalentes a su versión comercial, ya que poseen el mismo principio activo.

En programación, un algoritmo es genérico si su código no depende del tipo de los datos. Por ejemplo, en un lenguaje con pasaje de parámetros por referencia, el siguiente código intercambia los valores de las variables a y b, cualquiera sea el tipo T.

```
intercambiar (a,b:T) {  
  aux:T;  
  aux=a; a = b; b = aux;  
}
```

En numerosas aplicaciones es posible representar los datos con estructuras que pueden ser procesadas por algoritmos genéricos, esto es, el código no depende del tipo de las componentes de la estructura. Por ejemplo, el algoritmo para buscar el mayor elemento en una estructura de datos, puede generalizarse en un patrón que solo requiere que exista una relación de orden entre los elementos.

La genericidad es un recurso poderoso porque favorece la extensibilidad y la reusabilidad. Los datos de aplicaciones muy diferentes pueden modelarse con estructuras que serán creadas, modificadas y procesadas sin considerar el tipo de las componentes. La extensibilidad reduce el impacto de los cambios. Las modificaciones con frecuencia pueden resolverse definiendo nuevas clases específicas, sin necesidad de cambiar las que ya han sido desarrolladas y verificadas. La reusabilidad evita escribir el mismo código repetidamente acelerando el proceso de desarrollo.

Genericidad y Programación Orientada a Objetos

La programación orientada a objetos tiene como principal objetivo favorecer la confiabilidad, reusabilidad y extensibilidad del software. Adoptar el enfoque propuesto por la programación orientada a objetos implica:

- En la etapa de diseño reducir la complejidad en base a la descomposición del problema en piezas más simples, a partir de un conjunto de clases relacionadas entre sí.
- En la etapa de implementación utilizar un lenguaje que permita retener las relaciones entre las clases y encapsular su representación interna.

La abstracción de datos y el encapsulamiento permiten que una o más clases usen los servicios provistos por otra clase considerando sólo su comportamiento, sin tener en cuenta cómo lo implementa. La herencia permite aumentar el nivel de abstracción mediante un proceso de clasificación en niveles.

La herencia puede utilizarse para modelar genericidad, como describiremos en la siguiente sección. Alternativamente la genericidad puede modelarse usando polimorfismo paramétrico. El polimorfismo paramétrico en Java es limitado y tiene varias restricciones.

Clases genéricas

Una **clase genérica** es aquella que encapsula a una estructura cuyo comportamiento es independiente del tipo de sus elementos.

En Java es posible definir una clase `Fila` que encapsula un arreglo de elementos y brinda un método `esCreciente()` que retorna verdadero sí y sólo sí los elementos están ordenados de forma creciente. Esto es, dada la clase:

```
class Fila {
private Elemento [] f;
public Fila(int n){
    f = new Elemento[n];
}
...
}
```

La implementación del método puede ser:

```
public boolean esCreciente (){
/*Requiere que todas las posiciones del arreglo estén ligadas y que
el arreglo tenga al menos dos elementos*/
    int i=0; boolean es=true;
    while (i<f.length-1 && es){
        es = f[i].menor(f[i+1]);
        i++;
    }
    return es;
}
```

El código del método `esCreciente` es independiente del tipo de los elementos de la fila `f`. Para decidir si los elementos de una estructura lineal están ordenados en forma creciente, basta comparar cada elemento con el que le sigue hasta encontrar uno mayor que el que le sigue o llegar al anteúltimo. Esto es, se compara el primero con el segundo, el segundo con el tercero y así siguiendo mientras el elemento de la posición `i` del arreglo sea menor que el

elemento de la posición $i+1$ y además i sea menor a la cantidad de elementos del arreglo menos 1.

La implementación de `esCreciente` solo requiere que la clase `Elemento` brinde el servicio `menor`. Es decir, el programador puede implementar el método `esCreciente` conociendo exclusivamente la signature del método `menor`.

Si la clase `Elemento` brinda además el servicio `equals`, es posible definir un método genérico en la clase `Fila` para contar la cantidad de elementos del arreglo equivalentes a `e`.

```
public int contar (Elemento e){
/*Requiere que todas las posiciones del arreglo estén ligadas*/
    int cont =0;
    for (int i=0;i<f.length;i++)
        if (f[i].equals(e)) cont++;
    return cont;}

```

El método `contar` es independiente del tipo de los elementos, siempre que la clase `Elemento` brinde el servicio `equals`. El diagrama de la clase `Fila` puede ser entonces:

Fila
<<Atributo de instancia>> f: Elemento[]
<<Constructor>> Fila(n:entero) <<Comandos>> insertar(e:Elemento,p:entero) <<Consulta>> posicionValida(p:entero):boolean todasLigadas():boolean esCreciente():boolean contarElementos(e:Elemento):entero

- `Fila(n:entero)` Requiere n mayor a 0.
- `insertar(e:Elemento,p:entero)` Requiere p válida.
- `todasLigadas()` boolean Retorna true si todas las posiciones de la fila están ligadas.
- `esCreciente():boolean` Retorna true si los elementos de la fila están ordenados de menor a mayor. Requiere que todas las posiciones del arreglo estén ligadas y que contenga al menos dos elementos.
- `contarElementos(e:Elemento):entero` Retorna la cantidad de elementos equivalentes a `e`.

Y la implementación:

```
class Fila {
//Atributo de instancia
private Elemento [] f;
//Constructor
public Fila(int n){
    f = new Elemento[n];}
//Comandos
public void insertar (Elemento e, int p){
//Requiere p válida

```

```

    f[p] = e;}
//Consultas
public boolean posicionValida(int p){
    return p >= 0 && p < f.length; }
public boolean todasLigadas(){
    boolean todas=true;
    while (i<f.length && todas){
        todas = f[i] != null;
        i++; }
    return todas;}
public boolean esCreciente (){
    /*Retorna true si los elementos de la fila están ordenados de menor
    a mayor. Requiere que todas las posiciones del arreglo estén ligadas
    y que contenga al menos dos elementos*/
    int i=0; boolean es=true;
    while (i<f.length-1 && es){
        es = f[i].menor(f[i+1]);
        i++; }
    return es;}
public int contarElementos (Elemento e){
    /* Retorna la cantidad de elementos equivalentes a e.*/
    int cont =0;
    if (e !=null)
        for (int i=0;i<f.length;i++)
            if (f[i] != null && f[i].equals(e)) cont++;
    return cont;}}

```

Un objeto de clase `Fila` brinda servicios `posicionValida`, `todasLigadas`, `insertar`, `esCreciente` y `contarElementos`. Ninguno de los servicios depende del tipo de los elementos. Es decir, podemos implementar la clase `Fila` sin saber si va a ser utilizada para mantener referencias a objetos de clase `PresionArterial`, `Racional` o `Robot`. Lo que sí sabemos es que los elementos van a ser de un mismo tipo, para el cual tienen que estar definidos los métodos `equals` y `menor`.

La especificación de `contarElementos` no indica que la clase cliente sea responsable de garantizar que el parámetro está ligado, de modo que el método realiza ese control, para evitar errores de ejecución. También podría prevenirse este error si el método `equals` asumiera el compromiso de realizar el control.

Como la implementación de estos servicios no se conoce en el momento de crear la clase `Elemento`, los definimos como abstractos y por lo tanto la clase misma es abstracta:

```

abstract class Elemento {
    abstract boolean equals(Elemento e);
    abstract boolean menor (Elemento e);}

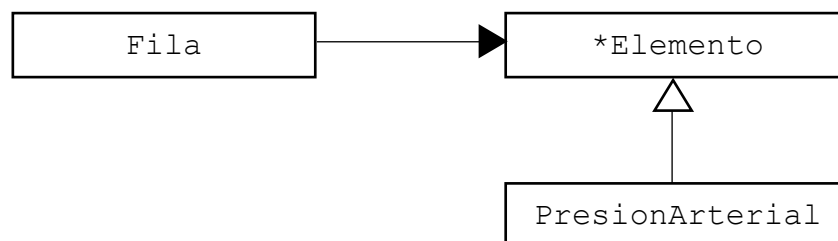
```

Una vez implementada la clase genérica `Fila` podemos usarla para crear estructuras con elementos de clase `PresionArterial`, `Racional` o `Robot` siempre que cada una de ellas extienda a la clase `Elemento` e implemente los métodos abstractos `equals` y `menor`.

Por ejemplo, en `PresionArterial`, `equals` retorna `true` sí y sólo sí los dos objetos que se comparan computan el mismo valor para el pulso. La implementación parcial de `PresionArterial` es:

```
class PresionArterial extends Elemento {
//Valores representados el milímetros de mercurio
//Atributos de clase
private static final int umbralMax=120;
private static final int umbralMin=80;
//Atributos de instancia
private int maxima;
private int minima;
//Constructor
public PresionArterial(int ma,int mi){
//Requiere ma > mi
    maxima = ma;
    minima = mi;}
//Consultas
public int obtenerPulso(){
    return maxima-minima;}
public boolean equals(Elemento e){
//Requiere e ligada y de clase PresionArterial
    return obtenerPulso() == ((PresionArterial) e).obtenerPulso();}
public boolean menor(Elemento e){
//Requiere e ligada y de clase PresionArterial
    return obtenerPulso() < ((PresionArterial) e).obtenerPulso ();}}
```

La clase `PresionArterial` extiende a la clase abstracta `Elemento` e implementa los métodos `equals` y `menor`, considerando que la comparación se realiza de acuerdo al pulso.



Como `equals` y `menor` redefinen al método definido en la clase `Elemento`, el parámetro tiene que ser de tipo `Elemento`, aun cuando lo que se van a comparar son dos objetos de clase `PresionArterial`. El casting justamente asegura que el mensaje `obtenerPulso()` va a poder ligarse a un método provisto para `e`.

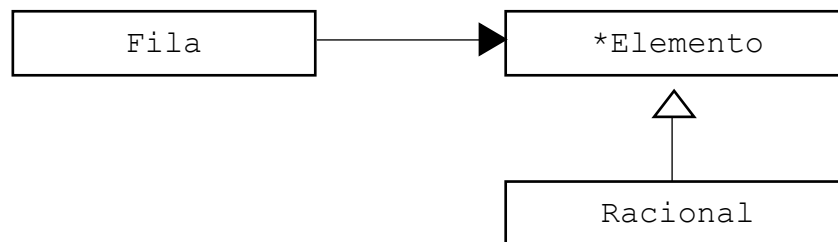
En un método de una clase `Hospital` que modele el registro de mediciones de presión arterial de un paciente, la clase `Fila` puede usarse como sigue:

```
class Hospital{
...
    Fila registro = new Fila (3);
    registro.insertar(new PresionArterial (7,13));
    registro.insertar(new PresionArterial (7,14));
    registro.insertar(new PresionArterial (7,15));
    if (registro.esCreciente())
        ...
}
```

La clase cliente crea un objeto `Fila` e inserta elementos que son instancias de una clase específica que deriva de la clase `Elemento`. En este ejemplo, los tres elementos de `registro` son de clase `PresionArterial`. Dos objetos de clase `PresionArterial` son **comparables**,

de modo que es perfectamente válido comparar el primer valor con el segundo, el segundo con el tercero y así siguiendo. La comparación se realiza usando el servicio `menor` provisto por la clase `PresionArterial`.

La clase `Fila` puede representar una fila de números racionales. Gráficamente:



La implementación parcial de `Racional` es:

```

class Racional extends Elemento{
//El denominador es siempre mayor a 0
//Atributo de instancia
private int numerador,denominador;
//Constructor
public Racional(int n,int d){
//Requiere d > 0
    numerador = n;
    denominador = d;}
//Consultas
public int obtenerNumerador(){
    return numerador;}
public int obtenerDenominador(){
    return denominador;}
public boolean equals(Elemento e){
//Requiere e ligada y de clase Racional
    Racional r = (Racional) e;
    return numerador*r.obtenerDenominador() ==
        r.obtenerNumerador()*denominador ; }
public boolean menor(Elemento e){
//Requiere e ligada y de clase Racional
    Racional r = (Racional) e;
    return numerador*r.obtenerDenominador() <
        r.obtenerNumerador()*denominador ; }}
    
```

Nuevamente, el tipo del parámetro en los servicios `equals` y `menor` debe ser `Elemento`, de modo que el método quede redefinido. Si el parámetro de una de estas consultas fuera de clase `Racional`, el servicio quedaría sobrecargado y no redefinido. El compilador reportaría entonces un error porque la `Racional` no implementa el método abstracto.

La especificación indica que es responsabilidad de la clase cliente controlar que el parámetro `e` es una variable ligada y que está ligada a un objeto de clase `Racional`. El diseñador podría decidir que sea la clase `Racional` quien debe asumir estos compromisos o incluso no especificarlo explícitamente. En este último caso como programadores, somos responsables de establecer controles que eviten errores de ejecución.

Como el método `obtenerDenominador()` no está definido para la clase `Elemento`, es necesario garantizar que va a estar provisto para el parámetro `e`. Para que los servicios

`equals` y `menor` queden redefinidos en la clase derivada, el parámetro debe ser de clase `Elemento`.

La clase `Fila` puede ser usada entonces, por ejemplo, en una aplicación matemática:

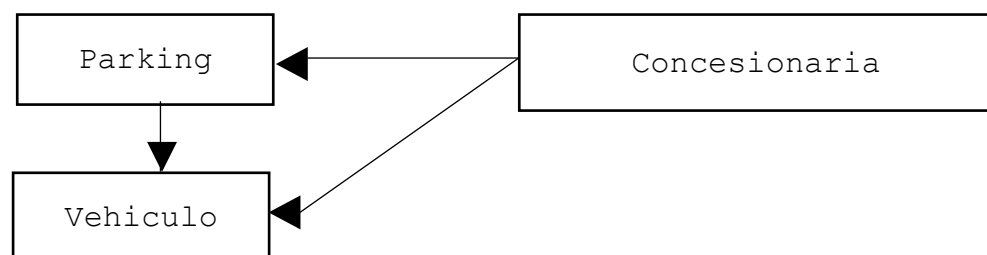
```
class Simulador{
...
Racional r1,r2,r3,r4;
    r1=new Racional(1,1);
    r2=new Racional(0,1);
    r3=new Racional (-1,1);
    r4=new Racional (1,2);
    Fila f = new Fila (4);
    f.insertar(r1);
    f.insertar(r2);
    f.insertar(r3);
    f.insertar(r4);
    if (f.todosLigados() && f.esCreciente())
...}
```

En este ejemplo en la fila `f` se insertan objetos de clase `Racional`, todos los elementos son instancias de `Racional`. Aunque `f` es una estructura genérica, el casting en los métodos `equals` y `esMenor` relaja el control del compilador sin provocar riesgos, porque en la aplicación `Simulador` los elementos de un objeto de clase `Fila` son todos de clase `Racional`. Análogamente en la aplicación `Hospital`, todos los elementos del objeto `registro` son de clase `PresionArterial`.

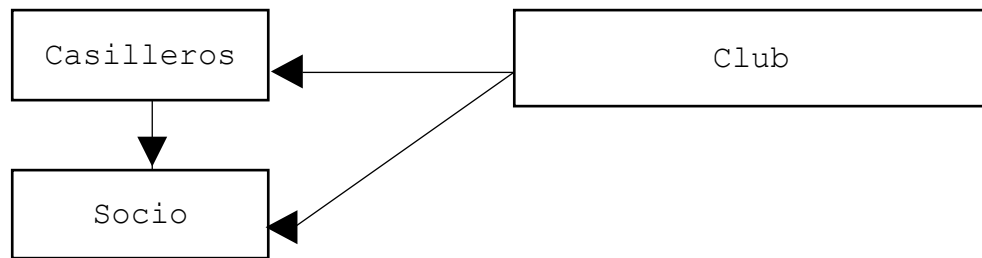
Generalización y reuso

En los capítulos anteriores se han presentado patrones de algoritmos que podían reusarse en aplicaciones muy diferentes. Al analizar el diagrama de clases de distintos sistemas es posible observar que un conjunto de servicios brindan una funcionalidad equivalente, de modo que se favorece el reuso definiendo una clase genérica que los incluya.

A continuación se retoma el caso de estudio referido a una concesionaria de autos que dispone de un estacionamiento en el cual se ubican los vehículos en venta. La clase `Parking` encapsula una **tabla** de elementos de tipo `Vehiculo`, representada a través de un arreglo parcialmente ocupado. Cada vehículo se asigna a una unidad particular del estacionamiento, aunque puede haber algunas libres. Gráficamente:



También es posible retomar el caso de un club que dispone de un conjunto de casilleros que alquila a sus socios para guardar sus pertenencias. Cada casillero se asigna a un socio particular, aunque puede haber algunos libres. Cada socio puede alquilar varios casilleros. Gráficamente:



Los diagramas de las clases **Parking** y **Casilleros** son:

Parking	Casilleros
<<Atributos de instancia>> t: [] Vehiculo	<<Atributos de instancia>> t: [] Socio
<<Constructor>> Parking (max:entero) <<Comandos>> estacionar(v:Vehiculo, p:entero) estacionar(v:Vehiculo) salir(p:entero) salir(v:Vehiculo) <<Consultas>> maxUnidades():entero unidadesOcupadas():entero unidadOcupada(p:entero):boolean hayUnidadOcupada():boolean estaLleno():boolean estaVehiculo(v:Vehiculo):boolean existeUnidad(p:entero):boolean recuperaVehiculo(p:entero): Vehiculo valuacionTotal():real menorKm():Vehiculo cantModelo(mod:entero):entero	<<Constructor>> Casilleros (max:entero) <<Comandos>> alquilar(s:Socio,p:entero) alquilar(s:Socio) liberar(p:entero) liberar(s:Socio) <<Consultas>> maximo():entero ocupados():entero casilleroOcupado(p:entero): boolean hayCasilleroOcupado():boolean todoLleno():boolean tieneCasillero(s:Socio):boolean existeCasillero(p:entero): boolean recuperaSocio(p:entero):Socio cantCasilleros(s:Socio):entero

En la clase **Parking** la funcionalidad de los servicios es:

- **estacionar(v:Vehiculo, p:entero)** asigna el vehículo **v** a la unidad **p** que asume válida y libre.
- **estacionar(v:Vehiculo)** asigna el vehículo **v** a la primera unidad libre. La clase Cliente es responsable de controlar que la tabla no esté llena y **v** no sea nulo
- **salir(p:entero)** asigna nulo a la unidad **p** que asume válida.
- **salir(v:Vehiculo)** busca una unidad ligada al vehículo **v** y le asigna nulo. Requiere que **v** no sea nulo.
- **maxUnidades():entero** computa la cantidad de unidades del estacionamiento .
- **unidadesOcupadas():entero** computa la cantidad de unidades ocupadas del estacionamiento.
- **unidadOcupada(p:entero):boolean** computa true si en la unidad **p** está estacionado un vehículo, asume **p** válida.
- **hayUnidadOcupada():boolean** computa true si al menos una unidad está ocupada.

- `estaLleno():boolean` computa true si todas las unidades están ocupadas.
- `estaVehiculo(v:Vehiculo):boolean` computa true si el vehículo está estacionado en una unidad.
- `existeUnidad(p:entero):boolean` computa true si `p` es una unidad del estacionamiento.
- `recuperaVehiculo(p:entero): Vehiculo` retorna el vehículo estacionado en la unidad `p`, que asume una unidad válida.
- `valuacionTotal():real` computa el valor que resulta de computar la suma de los valores de todos los vehículos.
- `menorKm():Vehiculo` retorna el vehículo estacionado con menor kilometraje.
- `cantModelo(mod:entero):entero` retorna la cantidad de vehículos estacionados del modelo `mod`.

La clase `Concesionaria` define un atributo de instancia de clase `Parking` para gestionar el estacionamiento:

```
class Concesionaria {
//Atributos de instancia
private Parking e;
...
public void gestionarUnidades() {

    Vehiculo v;
    int p;
    ...
    if (e.existeUnidad(p) && !e.unidadOcupada(p))
        e.estacionar(v,p);
    ...
    Vehiculo menor= e.menorKm(); }
...}
```

En la clase `Casilleros` la funcionalidad de los servicios es:

- `alquilar(s:Socio, p:entero)` asigna el socio `s` al casillero `p` que asume válido y libre.
- `alquilar(s:Socio)` asigna el socio `s` al primer casillero libre. La clase `Cliente` es responsable de controlar que la tabla no esté llena y `s` no sea nulo
- `liberar(p:entero)` asigna nulo al casillero `p` que asume válido.
- `liberar(s:Socio)` busca los casilleros asignados al socio `s` y les asigna nulo. Requiere que `s` esté ligado.
- `maximo():entero` computa la cantidad total de casilleros.
- `ocupados():entero` computa la cantidad de casilleros ocupados.
- `casilleroOcupado(p:entero):boolean` computa true si el casillero `p` está asignado a un socio, asume `p` válido.
- `hayCasilleroOcupado():boolean` computa true si al menos un casillero está ocupado.
- `todoLleno():boolean` computa true si todos los casilleros están ocupados.

- `tieneCasillero(s:Socio):boolean` computa true si el socio `s` tiene al menos un casillero asignado.
- `existeCasillero(p:entero):boolean` computa true si `p` denota un casillero válido.
- `recuperaSocio(p:entero):Socio` retorna el socio que tiene asignado el casillero `p`, asumiendo `p` válido.
- `cantCasilleros(s:Socio):entero` computa la cantidad de casilleros ligados al socio `s`.

Una implementación parcial de la administración de los casilleros en la clase `Club` puede ser:

```
class Club{
//Atributos de instancia
private Casilleros c;
...
public void administrar(){
    Socio s;
    int p;
    ...
    if (c.existeCasillero(p) && !c.casilleroOcupado(p))
        c.alquilar(s,p);
    ...
    int cant = c.cantCasilleros(s); }
...}
```

Se puede observar que algunos de los servicios de la clase `Casillero` tienen una funcionalidad equivalente a un servicio de `Parking`.

Al analizar la implementación del comando `estacionar`:

```
public void estacionar (Vehiculo v, int p) {
/*Asigna el vehiculo v a la unidad p, requiere que exista la unidad
y esté libre*/
    T[p] = v; }
```

se nota que aunque se trata de aplicaciones diferentes, el comando `alquilar` brinda la misma funcionalidad:

```
public void alquilar (Socio s, int p) {
/*Asigna el socio s al casillero p, requiere que exista el casillero
y esté libre*/
    T[p] = s; }
```

En Java toda clase, en particular `Socio` y `Vehiculo`, hereda implícitamente de la clase `Object`. Como el código de `estacionar` y de `alquilar` no depende del tipo de las componentes, puede generalizarse definiendo un comando que reciba un parámetro de clase `Object`:

```
public void insertar (Object e, int p) {
/*Asigna el parámetro e a la posición p, requiere que exista la
posición y esté libre*/
    T[p] = e; }
```

Análogamente, el método `estacionar` inserta un vehículo en la primera unidad libre:

```
public void estacionar (Vehiculo v) {
/*Busca la primera unidad libre y asigna v. La clase Cliente es
responsable de controlar que la tabla no esté llena y v no sea
nulo*/
    int i = 0;
    while (T[i] != null)
        i++;
    T[i] = v;}

```

Tiene una funcionalidad equivalente al servicio `alquilar` que asigna el socio `s` al primer casillero libre:

```
public void alquilar (Socio s) {
/*Busca el primer casillero libre y asigna el socio s. La clase
Cliente es responsable de controlar que la tabla no esté llena y s
no sea nulo*/
    int i = 0;
    while (T[i] != null)
        i++;
    T[i] = s;}

```

La solución puede generalizarse para cualquier parámetro de clase `Object` ya que el código no depende del tipo de las componentes:

```
public void insertar (Object e) {
/*Busca la primera posición libre y asigna e. La clase Cliente es
responsable de controlar que la tabla no esté llena y e no sea
nulo*/
    int i = 0;
    while (T[i] != null)
        i++;
    T[i] = e;}

```

Si las búsquedas comparan la **identidad de los elementos** se puede generalizar el comportamiento de la consulta `estaVehiculo` en la clase `Parking`:

```
public boolean estaVehiculo (Vehiculo v){
/*Decide si el vehículo v está estacionado en alguna unidad */
    int i = 0; boolean esta = false;
    while (i < T.length && !esta ){
        esta = T[i] != null && T[i] == v ;
        i++; }
    return esta;}

```

También se puede generalizar la consulta `tieneCasillero` en la clase `Casilleros`.

```
public boolean tieneCasillero (Socio s){
/*Decide si el socio s tiene asignado algún casillero */
    int i = 0; boolean esta = false;
    while (i < T.length && !esta ){
        esta = T[i] != null && T[i] == s ;
        i++; }
    return esta;}

```

Los métodos `estaVehiculo` y `tieneCasillero` brindan una funcionalidad equivalente y las responsabilidades son las mismas.

Es importante considerar cuidadosamente la funcionalidad y las responsabilidades porque puede haber variaciones. En la concesionaria cada vehículo puede estar estacionado a lo sumo a una unidad, en el club en cambio un socio puede estar asignado a varios casilleros. El código:

```
public void salir(Vehiculo v){
/*Busca una unidad asignada al vehículo y si existe asigna nulo.
Requiere que v está ligado*/
    int i = 0; boolean esta=false;
    while (i < T.length && !esta){
        if (T[i] == v) esta = true ;
        else i++; }
    if (esta) T[i] = null;}
```

No es análogo a la consulta `liberar` en la clase `Casilleros`:

```
public void liberar(Socio s){
/*Busca todos los casilleros asignados al socio s y les asigna
nulo. Requiere que v está ligado*/
    int i = 0; boolean esta=false;
    while (i < T.length ){
        if (T[i] == s) T[i] = null;
        i++;}}
```

De modo que estos comandos no son generalizables.

Servicios generales de una clase

Una clase genérica brinda un conjunto de servicios generales cuya implementación es independiente del tipo de las componentes de la estructura. La clase genérica puede ser usada para modelar diferentes aplicaciones.

A partir de las dos aplicaciones presentadas en la sección anterior, se propone el diseño de una clase genérica `Tabla` de acuerdo al siguiente diagrama:

Tabla
<<Atributo de instancia>> T: [] Object
<<Constructor>> Tabla (max:entero) <<Comandos>> insertar(elem:Object,p:entero) insertar(elem:Object) eliminar(p:entero) <<Consultas>> maxElem():entero cantElem():entero estaLlena():boolean hayElem():boolean estaElem(elem:Object):boolean existePosicion(p:entero):boolean posicionOcupada(p:entero):boolean recuperaElem(p:entero):Object

Con las siguientes funcionalidades y responsabilidades:

- `insertar(elem:Object, p:entero)` asigna el elemento `elem` a la posición `p` que asume válida y libre.
- `insertar(elem:Object)` asigna el elemento `elem` a la primera posición libre. La clase Cliente es responsable de controlar que la tabla no esté llena y e no sea nulo.
- `eliminar(p:entero)` asigna nulo a la posición `p` que asume válida.
- `maxElem():entero` computa la cantidad de posiciones de la tabla.
- `cantElem():entero` computa la cantidad de posiciones ligadas.
- `hayElem():boolean` computa true si al menos una posición está ocupada.
- `estaLlena():boolean` computa true si todas las posiciones de la tabla están ocupadas.
- `estaElem(elem:Object):boolean` computa true si el elemento `elem` está asignado al menos a una posición.
- `existePosicion(p:entero):boolean` computa true si `p` denota una posición válida en la tabla.
- `posicionOcupada(p:entero):boolean` computa true si la posición `p` está ligada, asume `p` válida.
- `recuperaElem(p:entero):Object` retorna el elemento asignado a la posición `p`, que asume válida.

Y la implementación es:

```
class Tabla {
protected Object[] T;
//Constructor
public Tabla(int max){
    T = new Object[max];}
//Comandos
public void insertar(Object elem, int p) {
/*Asigna elem a la posicion p, requiere que p sea válida y esté
libre */
    T[p] = elem;}

public void insertar (Object e) {
/*Busca la primera posición libre y asigna e. La clase Cliente es
responsable de controlar que la tabla no esté llena y e no sea
nulo*/
    int i = 0;
    while (T[i] != null)
        i++;
    T[i] = e;}

public void eliminar(int p){
/*Elimina el elemento de la posicion p, requiere que p sea una
posición válida*/
    T[p] = null;}
//Consultas
public int maxElem(){
    return T.length;}
public int cantElem(){
//Retorna la cantidad de referencias ligadas
```

```

    int cant=0;
    for (int i=0; i<T.length;i++)
        if (T[i] != null) cant++;
    return cant;}
public boolean posicionOcupada(int i){
    return T[i] != null;}
public boolean hayElem(){
    /*Retorna true sí y sólo sí al menos una referencia está ligada*/
    int i = 0; boolean hay=false;
    while (i < T.length && !hay){
        if (T[i] != null) hay = true ;
        i++;}
    return hay;}
public boolean estaLlena(){
    int i = 0; boolean hay=false;
    while (i < T.length && !hay){
        if (T[i] == null) hay = true ;
        i++;}
    return !hay;}
public boolean estaElem(Object elem){
    /*Retorna true sí y sólo si existe una posición ligada a elem*/
    int i = 0; boolean esta=false;
    while (i < T.length && !esta){
        esta = T[i] == elem;
        i++;}
    return esta;}
public boolean existePosicion (int p) {
    /*Retorna true sí y sólo sí p denota una posición válida de la
    tabla*/
    return p >= 0 && p < T.length;}
public Object recuperarElemento(int p) {
    /* Retorna el elemento que ocupa la posición p en la tabla Requiere
    que p sea una posición válida*/
    return T[p];}
}

```

El conjunto de servicios provistos por la clase `Tabla` es independiente del tipo de las componentes.

Servicios específicos de una clase

En muchas aplicaciones es posible reusar una clase genérica, pero es necesario extenderla con otra que brinde el comportamiento específico.

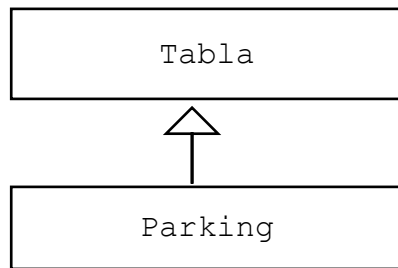
Algunos de los servicios provistos por la clase `Parking` son específicos de la aplicación:

- La salida de un vehículo del estacionamiento.
- El cómputo de cuántos vehículos mantiene el estacionamiento.
- El cómputo de la valuación total de los vehículos del estacionamiento.
- La determinación del vehículo con menor kilometraje.

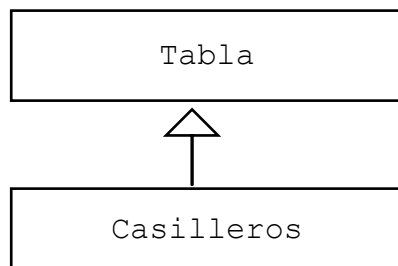
De manera análoga, la clase `Casilleros` ofrece algunos servicios específicos, cuyo código depende del tipo de las componentes:

- La liberación de todos los casilleros de un socio.
- El cómputo de la cantidad de casilleros asignados a un socio.

La clase `Parking` extiende a la clase `Tabla`:



De manera análoga `Casilleros` extiende a la clase `Tabla`:



Las clases derivadas definen los servicios que son específicos de cada aplicación y heredan los servicios generales provistos por la clase base `Tabla`:

Parking	Casilleros
<pre> <<Constructor>> Parking (max:entero) <<Comandos>> salir(v:Vehiculo) <<Consultas>> valuacionTotal():real menorKm():Vehiculo cantModelo(mod:entero):entero </pre>	<pre> <<Constructor>> Casilleros (max:entero) <<Comandos>> liberar(s:Socio) <<Consultas>> cantCasilleros(s:Socio):entero </pre>
Requiere que la clase cliente solo inserte elementos que son instancia de Vehiculo	Requiere que la clase cliente solo inserte elementos que son instancia de Socio

En este caso, ninguna de las clases especializadas agrega atributos. Cada uno de los servicios brinda la funcionalidad descrita anteriormente a partir del acceso y recorrido de la estructura definida en la clase genérica.

La implementación de la clase `Casilleros` es entonces:

```

class Casilleros extends Tabla {
public Casilleros (int max){
    super(max);}
public void liberar (Socio s){
/*Asigna nulo a todos los casilleros ligados al socio s*/
    int i = 0;
    while (i < T.length){
        if (T[i] == s)
            T[i] = null ;
        i++;}}
public int cantCasilleros (Socio s){

```

```
/*Computa la cantidad de casilleros asociados al socio s*/
int i = 0; int cont = 0;
while (i < T.length){
    if (T[i] == s)
        cont++;
    i++;}
return cont;}
}
```

Como antes, la clase `Club` define un atributo de instancia de clase `Casilleros` y modelará la administración de casilleros.

```
class Club{
//Atributos de instancia
private Casilleros c;
...
public void administrar(){
    Socio s;
    int p;
    ...
    if (c.existePosicion(p) && !c.posicionOcupada(p))
        c.insertar (s,p);
    ...
    int cant = c.cantCasilleros(s); }
...}
```

El objeto ligado a la variable `c` recibe los mensajes `insertar` y `cantCasilleros`, el primero definido en la clase base `Tabla` y el segundo en la clase derivada `Casilleros`. La definición de una clase genérica permite reusar el código en diferentes aplicaciones, en este caso la implementación de una tabla.